

Universidad Nacional Abierta y a Distancia – UNAD

Escuela de Ciencias Básicas, Tecnología e Ingeniería – ECBTI

Programa: Ingeniería de Sistemas

Fase 5 – Evaluación Final POA

Problema 3: Auditoría de Inventario y Reabastecimiento

Curso: Fundamentos de Programación

Código del Curso: 213022

Elaborado por:

Emmanuel Orellano Villazón

Tutor: Eder L. Cosme

Soledad, Atlántico, Colombia – Mayo de 2026

1. Análisis del Problema

El Problema 3 solicita construir una herramienta de software que permita auditar el estado de un inventario y generar una lista de reabastecimiento. La solución debe procesar una estructura de datos matricial (lista anidada) que contiene, por cada artículo: su código identificador, nombre descriptivo, stock actual disponible y el stock mínimo requerido para la operación normal.

La lógica central del programa consiste en comparar, para cada artículo, el stock actual con el mínimo requerido. Si el stock actual es insuficiente, se debe calcular la diferencia exacta de unidades a solicitar. Si el stock es suficiente, no se genera pedido. El resultado final es un informe tabular completo y una lista consolidada de pedidos.

Entradas del sistema:

- Una matriz (lista anidada en Python) con al menos 5 artículos.
- Cada fila contiene: [Código, Nombre, Stock Actual, Stock Mínimo Requerido].

Procesos requeridos:

- Recorrer toda la matriz con un ciclo repetitivo (for).
- Por cada artículo, invocar la función `calcular_pedido()` que implementa la lógica condicional.
- Clasificar el estado del stock con otra función usando `if/elif/else`.
- Acumular los artículos con pedido en una lista separada.

Salidas esperadas:

- Tabla completa con todos los artículos, sus stocks y la cantidad a pedir.
- Lista de pedidos con solo los artículos que requieren reabastecimiento.
- Total de unidades a pedir.

2. Tabla de Requerimientos

La siguiente tabla detalla cada requisito funcional del sistema, su descripción precisa, prioridad y el elemento del código que lo implementa.

ID	Requisito	Descripción	Prioridad	Implementación
RF-01	Datos iniciales	Crear una lista anidada (matriz) con mínimo 5 artículos	Alta	<code>inventario[][]</code>
RF-02	Función principal	Implementar <code>calcular_pedido(stock_actual, stock_minimo)</code>	Alta	<code>calcular_pedido()</code>

RF-03	Lógica condicional	if stock_actual < stock_minimo → devolver diferencia; else → devolver 0	Alta	if / else
RF-04	Clasificación estado	Función clasificar_estado() con if/elif/else para etiqueta visual	Media	clasificar_estado()
RF-05	Estructura repetitiva	Ciclo for para recorrer toda la matriz de inventario	Alta	for artículo in inventario
RF-06	Informe tabular	Mostrar todos los artículos con formato alineado y legible	Alta	generar_informe()
RF-07	Lista de pedidos	Filtrar y mostrar solo los artículos que requieren pedido	Alta	mostrar_lista_pedidos()
RF-08	Total unidades	Calcular suma total de unidades a pedir	Media	ciclo for acumulador
RF-09	Buenas prácticas	Comentarios, nombres descriptivos, funciones separadas, main()	Alta	Todo el código
RF-10	Punto de entrada	Usar if __name__ == '__main__': para ejecutar el programa	Media	main()

Nota. Elaboración propia basada en el banco de problemas Fase 5, UNAD (2026).

3. Algoritmo Paso a Paso

A continuación se presenta el algoritmo en pseudocódigo que describe la lógica del programa antes de su implementación en Python:

INICIO DEL PROGRAMA

```
1. DEFINIR inventario como lista anidada con 7 artículos
   Cada artículo = [Código, Nombre, Stock_Actual, Stock_Mínimo]
```

```
2. FUNCIÓN calcular_pedido(stock_actual, stock_minimo):
   2.1 SI stock_actual < stock_minimo:
       RETORNAR (stock_minimo - stock_actual)
   2.2 SINO:
       RETORNAR 0
```

```
3. FUNCIÓN clasificar_estado(stock_actual, stock_minimo):
   3.1 SI stock_actual == 0: RETORNAR 'CRÍTICO'
   3.2 SINO SI stock_actual < min: RETORNAR 'BAJO'
   3.3 SINO SI stock_actual == min: RETORNAR 'JUSTO'
   3.4 SINO: RETORNAR 'OK'
```

```
4. FUNCIÓN generar_informe(inventario):
   4.1 Imprimir encabezado de tabla
   4.2 Inicializar lista_pedidos = []
```

```
4.3 PARA CADA artículo EN inventario:
    4.3.1 Extraer: codigo, nombre, stock_actual, stock_minimo
    4.3.2 cantidad = calcular_pedido(stock_actual, stock_minimo)
    4.3.3 estado = clasificar_estado(stock_actual, stock_minimo)
    4.3.4 Imprimir fila del artículo
    4.3.5 SI cantidad > 0: AGREGAR a lista_pedidos
4.4 RETORNAR lista_pedidos
```

```
5. FUNCIÓN mostrar_lista_pedidos(lista_pedidos):
    5.1 SI lista_pedidos está vacía: imprimir mensaje OK
    5.2 SINO:
        5.2.1 PARA CADA pedido EN lista_pedidos:
            Imprimir nombre, cantidad, estado
        5.2.2 Calcular y mostrar TOTAL de unidades
```

```
6. FUNCIÓN main():
    6.1 lista_pedidos = generar_informe(inventario)
    6.2 mostrar_lista_pedidos(lista_pedidos)
```

```
7. SI __name__ == '__main__': LLAMAR main()
```

FIN DEL PROGRAMA

4. Diagrama de Flujo – Descripción Detallada

A continuación se describe el diagrama de flujo del programa. Para la sustentación, este diagrama debe dibujarse a mano o con una herramienta como draw.io o Lucidchart.

Flujo Principal (main)

INICIO → Definir inventario (lista anidada) → Llamar generar_informe(inventario) → Llamar mostrar_lista_pedidos(lista_pedidos) → FIN

Flujo de generar_informe()

INICIO FUNCIÓN → Imprimir encabezado → Inicializar lista_pedidos=[] → ¿Quedan artículos en inventario? → Sí: Extraer [codigo, nombre, stock_actual, stock_minimo] → Llamar calcular_pedido() → Llamar clasificar_estado() → Imprimir fila → ¿cantidad > 0? → Sí: Agregar a lista_pedidos → Volver a verificar artículos → NO (fin lista): Retornar lista_pedidos → FIN FUNCIÓN

Flujo de calcular_pedido()

INICIO FUNCIÓN → Recibir (stock_actual, stock_minimo) → ¿stock_actual < stock_minimo? → Sí: Calcular diferencia = stock_minimo – stock_actual → Retornar diferencia → NO: Retornar 0 → FIN FUNCIÓN

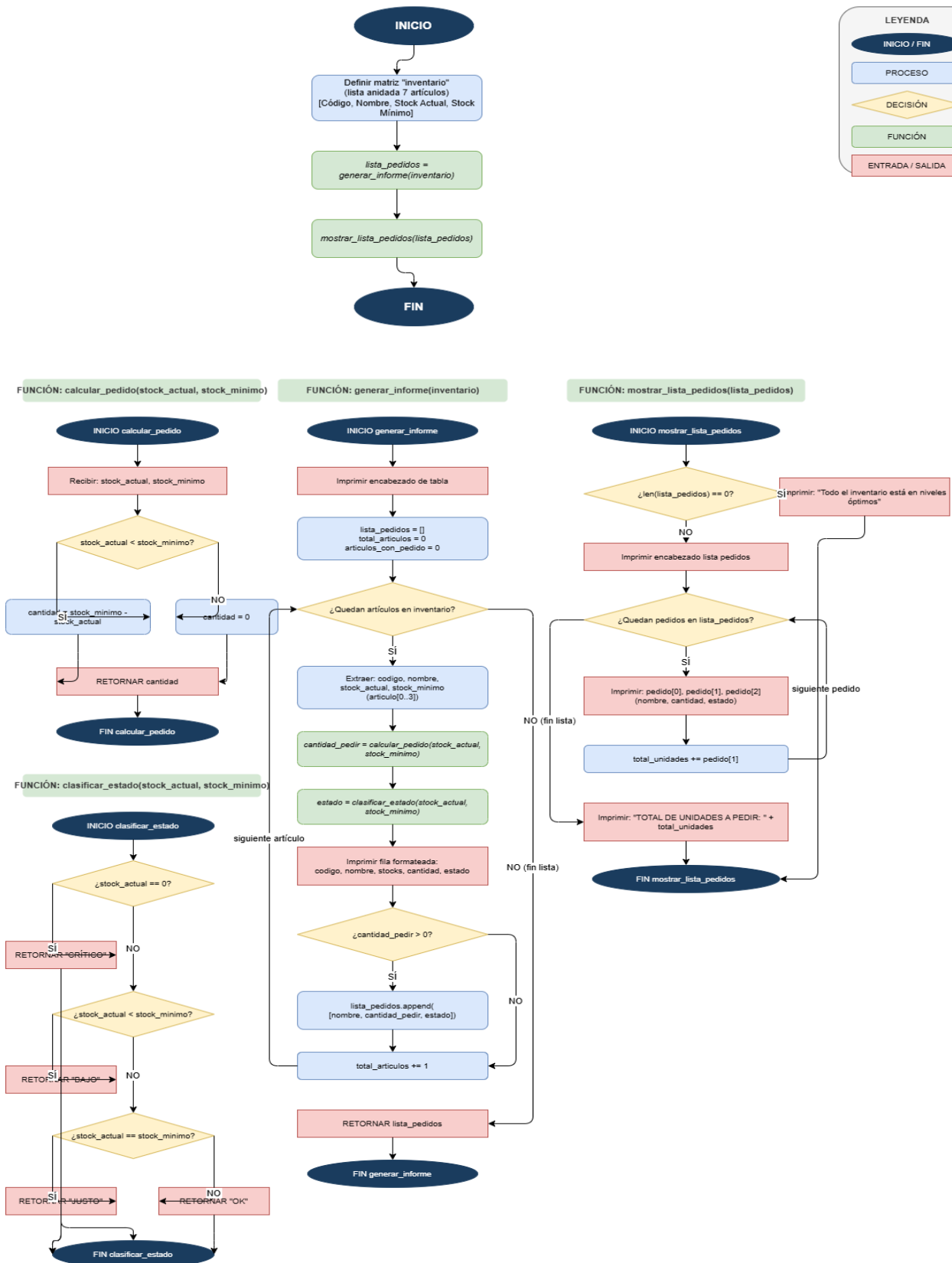
Flujo de clasificar_estado()

INICIO FUNCIÓN → ¿stock_actual == 0? → Sí: Retornar 'CRÍTICO' → NO: ¿stock_actual < stock_minimo? → Sí: Retornar 'BAJO' → NO: ¿stock_actual == stock_minimo? → Sí: Retornar 'JUSTO' → NO: Retornar 'OK' → FIN FUNCIÓN

Flujo de mostrar_lista_pedidos()

INICIO FUNCIÓN → ¿len(lista_pedidos) == 0? → Sí: Imprimir 'Todo OK' → NO: Imprimir encabezado → ¿Quedan pedidos? → Sí: Imprimir nombre, cantidad, estado → Volver → NO: Calcular total unidades → Imprimir total → FIN FUNCIÓN

DIAGRAMA DE FLUJO
 Sistema de Auditoría de Inventario y Reabastecimiento
 Emmanuel Orellano Villazón – UNAD – Fundamentos de Programación 213022



5. Prueba de Escritorio

La prueba de escritorio simula manualmente la ejecución del programa, trazando el valor de las variables en cada iteración del ciclo for para verificar que la lógica produce los resultados esperados antes de ejecutar el código.

#	Código	Nombre	S.Act	S.Mín	Llamada función	Condición	Operación	Estado	Pedido
1	ART-001	Teclado Mecánico	15	20	calcular_pedido(15, 20)	$15 < 20 \rightarrow \text{True}$	$20 - 15 = 5$	BAJO	5
2	ART-002	Monitor 24 pulg.	3	10	calcular_pedido(3, 10)	$3 < 10 \rightarrow \text{True}$	$10 - 3 = 7$	BAJO	7
3	ART-003	Mouse Inalámbrico	25	15	calcular_pedido(25, 15)	$25 < 15 \rightarrow \text{False}$	$\text{else} \rightarrow 0$	OK	0
4	ART-004	Cable HDMI 2m	2	30	calcular_pedido(2, 30)	$2 < 30 \rightarrow \text{True}$	$30 - 2 = 28$	BAJO	28
5	ART-005	Memoria RAM 8GB	8	10	calcular_pedido(8, 10)	$8 < 10 \rightarrow \text{True}$	$10 - 8 = 2$	BAJO	2
6	ART-006	Disco Duro 1TB	0	5	calcular_pedido(0, 5)	$0 < 5 \rightarrow \text{True}$	$5 - 0 = 5$	CRÍTICO	5
7	ART-007	Audífonos USB	12	12	calcular_pedido(12, 12)	$12 < 12 \rightarrow \text{False}$	$\text{else} \rightarrow 0$	JUSTO	0

Nota. S.Act = Stock Actual; S.Mín = Stock Mínimo. Elaboración propia.

Verificación de resultados:

- Artículos con pedido generado: ART-001 (5), ART-002 (7), ART-004 (28), ART-005 (2), ART-006 (5)
- Artículos sin pedido: ART-003 (stock OK), ART-007 (stock JUSTO)
- Total unidades a pedir: $5 + 7 + 28 + 2 + 5 = 47$ unidades ✓

6. Código Python Completo

El siguiente es el código fuente completo del programa, listo para ser copiado en un archivo .py y ejecutado con Python 3.x:

```
# =====
# SOLUCIÓN: AUDITORÍA DE INVENTARIO Y REABASTECIMIENTO
# Estudiante: Emmanuel Orellano Villazón
# Curso: Fundamentos de Programación - Código: 213022
# Universidad Nacional Abierta y a Distancia - UNAD
# =====

inventario = [
    ["ART-001", "Teclado Mecánico", 15, 20],
    ["ART-002", "Monitor 24 pulgadas", 3, 10],
    ["ART-003", "Mouse Inalámbrico", 25, 15],
    ["ART-004", "Cable HDMI 2m", 2, 30],
    ["ART-005", "Memoria RAM 8GB", 8, 10],
    ["ART-006", "Disco Duro 1TB", 0, 5],
    ["ART-007", "Audífonos USB", 12, 12],
]

def calcular_pedido(stock_actual, stock_minimo):
    """Calcula la cantidad exacta a pedir para un artículo."""
    if stock_actual < stock_minimo:
        cantidad_a_pedir = stock_minimo - stock_actual
        return cantidad_a_pedir
    else:
        return 0

def clasificar_estado(stock_actual, stock_minimo):
    """Clasifica el estado del stock del artículo."""
    if stock_actual == 0:
        return "CRÍTICO"
    elif stock_actual < stock_minimo:
        return "BAJO"
    elif stock_actual == stock_minimo:
        return "JUSTO"
    else:
        return "OK"

def generar_informe(inventario):
    """Recorre el inventario y genera el informe de auditoría."""
    print("=" * 75)
    print("      SISTEMA DE AUDITORÍA DE INVENTARIO - UNAD TECH STORE")
    print("=" * 75)
    print(f"{'Código':<10} {'Nombre':<25} {'S.Actual':>9} {'S.Mínimo':>9} {'A.Pedir':>8} {'Estado':<10}")
    print("-" * 75)
    lista_pedidos = []
    total_articulos = 0
```



```

articulos_con_pedido = 0
for articulo in inventario:
    codigo      = articulo[0]
    nombre      = articulo[1]
    stock_actual = articulo[2]
    stock_minimo = articulo[3]
    cantidad_pedir = calcular_pedido(stock_actual, stock_minimo)
    estado = clasificar_estado(stock_actual, stock_minimo)
    print(f"{codigo:<10} {nombre:<25} {stock_actual:>9} {stock_minimo:>9}
{cantidad_pedir:>8} {estado:<10}")
    if cantidad_pedir > 0:
        lista_pedidos.append([nombre, cantidad_pedir, estado])
        articulos_con_pedido += 1
    total_articulos += 1
print("-" * 75)
print(f"  Total artículos: {total_articulos}   |   Con pedido:
{articulos_con_pedido}")
return lista_pedidos

def mostrar_lista_pedidos(lista_pedidos):
    """Muestra solo los artículos que requieren reabastecimiento."""
    print("\n" + "=" * 50)
    print("          LISTA DE PEDIDOS A REALIZAR")
    print("=" * 50)
    if len(lista_pedidos) == 0:
        print("  Todo el inventario está en niveles óptimos.")
    else:
        print(f"   {'Artículo':<28} {'Cantidad':>8}   {'Estado'}")
        print("-" * 50)
        for pedido in lista_pedidos:
            print(f"   {pedido[0]:<28} {pedido[1]:>8}   {pedido[2]}")
        print("-" * 50)
        total_unidades = 0
        for pedido in lista_pedidos:
            total_unidades += pedido[1]
        print(f"  TOTAL DE UNIDADES A PEDIR: {total_unidades}")
        print("=" * 50)

def main():
    """Función principal: orquesta la ejecución del programa."""
    print("\n")
    lista_pedidos = generar_informe(inventario)
    mostrar_lista_pedidos(lista_pedidos)
    print("\n  Auditoría finalizada. Emmanuel Orellano Villazón - UNAD 2026\n")

if __name__ == '__main__':
    main()

```

The screenshot shows a Windows PowerShell terminal window on the left and a Sublime Text editor on the right. The terminal displays the output of running the 'inventario.py' script. The output includes a header for 'SISTEMA DE AUDITORÍA DE INVENTARIO - UNAD TECH STORE', a table of inventory items with columns for Código, Nombre, S.Actual, S.Mínimo, A.Pedir, and Estado, and a list of pending orders. The Sublime Text editor shows the Python code for the script, with comments explaining each line. The code includes a function to classify the state of an item based on its stock, a function to print the inventory table, and a function to show the list of pending orders.

7. Explicación Línea por Línea del Código

La siguiente tabla explica el propósito de cada línea o bloque relevante del código, detallando qué estructura de programación implementa y por qué cumple los criterios de la rúbrica:

Línea de Código	Explicación y criterio de rúbrica cubierto
# =====	Encabezado del programa con datos del estudiante y curso.
inventario = [["ART-001", "Teclado Mecánico", 15, 20], ["ART-002", "Monitor 24 pulgadas", 3, 10],	Inicio de la lista anidada (matriz). Cada elemento interno es una sublista. Fila 1: código, nombre, stock actual 15, stock mínimo 20. Requiere pedido. Fila 2: stock 3, mínimo 10. Stock muy bajo → pedido urgente.

["ART-003", "Mouse Inalámbrico", 25, 15],	Fila 3: stock 25, mínimo 15. Stock suficiente → no requiere pedido.
["ART-004", "Cable HDMI 2m", 2, 30],	Fila 4: stock 2, mínimo 30. Diferencia grande → pedido de 28 unidades.
["ART-005", "Memoria RAM 8GB", 8, 10],	Fila 5: stock 8, mínimo 10. Levemente por debajo → pedido de 2.
["ART-006", "Disco Duro 1TB", 0, 5],	Fila 6: stock 0 → estado CRÍTICO. Debe pedirse inmediatamente.
["ART-007", "Audífonos USB", 12, 12],	Fila 7: stock exactamente igual al mínimo → estado JUSTO, no pide.
]	Cierre de la lista anidada inventario.
def calcular_pedido(stock_actual, stock_minimo):	Definición de la función principal requerida. Recibe dos parámetros enteros.
"""Docstring: documenta la función."""	Documentación interna de la función (buena práctica profesional).
if stock_actual < stock_minimo:	Estructura condicional IF: evalúa si el stock actual es insuficiente.
cantidad_a_pedir = stock_minimo - stock_actual	Calcula la diferencia exacta: cuántas unidades faltan para cubrir el mínimo.
return cantidad_a_pedir	Retorna la cantidad calculada al módulo que llamó la función.
else:	ELSE: si la condición anterior es falsa (stock suficiente).
return 0	Retorna 0: no se necesita pedir nada para este artículo.
def clasificar_estado(stock_actual, stock_minimo):	Segunda función: clasifica visualmente el estado del stock.

<code>if stock_actual == 0:</code>	IF: caso más crítico, sin unidades disponibles.
<code> return "CRÍTICO"</code>	Retorna etiqueta CRÍTICO para stock en cero.
<code>elif stock_actual < stock_minimo:</code>	ELIF: stock por debajo del mínimo pero mayor a cero.
<code> return "BAJO"</code>	Retorna etiqueta BAJO.
<code>elif stock_actual == stock_minimo:</code>	ELIF: stock exactamente en el límite mínimo.
<code> return "JUSTO"</code>	Retorna etiqueta JUSTO.
<code>else:</code>	ELSE: stock supera el mínimo.
<code> return "OK"</code>	Retorna etiqueta OK: nivel de inventario saludable.
<code>def generar_informe(inventario):</code>	Función que recorre toda la matriz y genera el reporte tabular.
<code> print("=" * 75)</code>	Imprime línea separadora de 75 caracteres para el encabezado.
<code> print("SISTEMA DE AUDITORÍA...")</code>	Título del informe en consola.
<code> print(f'{"Código":<10} ...')</code>	Encabezado de columnas con formato f-string alineado.
<code> lista_pedidos = []</code>	Lista vacía que acumulará los artículos que requieren pedido.
<code> for articulo in inventario:</code>	CICLO FOR: itera sobre cada sublista (fila) de la matriz inventario.
<code> codigo = articulo[0]</code>	Extrae el código del artículo (posición 0 de la sublista).
<code> nombre = articulo[1]</code>	Extrae el nombre (posición 1).
<code> stock_actual = articulo[2]</code>	Extrae el stock actual (posición 2).
<code> stock_minimo = articulo[3]</code>	Extrae el stock mínimo requerido (posición 3).

<code>cantidad_pedir = calcular_pedido(stock_actual, stock_minimo)</code>	Llama a la función <code>calcular_pedido()</code> para este artículo.
<code>estado = clasificar_estado(stock_actual, stock_minimo)</code>	Llama a la función <code>clasificar_estado()</code> para obtener la etiqueta.
<code>print(f"{codigo:<10} ...")</code>	Imprime la fila formateada del artículo en la tabla.
<code>if cantidad_pedir > 0:</code>	IF: solo agrega a pedidos si la cantidad es mayor a cero.
<code>lista_pedidos.append([nombre, cantidad_pedir, estado])</code>	Agrega el artículo a la lista de pedidos como sublista.
<code>return lista_pedidos</code>	Retorna la lista de pedidos al módulo que llamó esta función.
<code>def mostrar_lista_pedidos(lista_pedidos):</code>	Función que imprime solo los artículos que necesitan reabastecimiento.
<code>if len(lista_pedidos) == 0:</code>	IF: verifica si la lista de pedidos está vacía.
<code>print(" Todo el inventario...")</code>	Mensaje si no hay pedidos necesarios.
<code>else:</code>	ELSE: sí hay pedidos que mostrar.
<code>for pedido in lista_pedidos:</code>	CICLO FOR: recorre cada elemento de la lista de pedidos.
<code>print(f" {pedido[0]:<28}...")</code>	Imprime nombre, cantidad y estado de cada pedido.
<code>total_unidades = 0</code>	Variable acumuladora para el total de unidades.
<code>for pedido in lista_pedidos:</code>	Segundo ciclo for para sumar todas las cantidades.
<code>total_unidades += pedido[1]</code>	Suma la cantidad de cada pedido al acumulador.
<code>def main():</code>	Función <code>main()</code> : punto de entrada y orquestador del programa.
<code>lista_pedidos = generar_informe(inventario)</code>	Llama a <code>generar_informe()</code> y recibe

	la lista de pedidos resultante.
<code>mostrar_lista_pedidos(lista_pedidos)</code>	Llama a <code>mostrar_lista_pedidos()</code> con la lista obtenida.
<code>if __name__ == '__main__':</code>	Guarda de entrada estándar de Python: solo ejecuta <code>main()</code> si se corre directamente.
<code>main()</code>	Llama a la función principal que inicia todo el programa.

8. Enlaces de entrega

Repositorio GitHub

<https://github.com/emmanuel231813-bit/programa-python>

Video de Sustentación

<https://youtu.be/HuWTqanJLSU>

9. Estructura del Repositorio GitHub

Para cumplir el cuarto criterio de evaluación, el repositorio debe ser público y estar correctamente estructurado. A continuación se detalla la organización ideal:

Nombre del repositorio

programa-python

Estructura de archivos

```
programa-python/  
├── inventario.py      ← Código fuente principal del programa  
├── README.md         ← Documentación del proyecto  
├── docs/  
│   └── informe.pdf   ← Informe académico completo en PDF
```

Pasos para crear el repositorio en GitHub

Paso 1 – Ingresar a github.com y crear una cuenta si no tienes una.

Paso 2 – Hacer clic en el botón '+' → 'New repository'.

Paso 3 – Escribir el nombre: programa-python.

Paso 4 – Marcar la opción 'Public' (obligatorio para que el tutor pueda verlo).

Paso 5 – Marcar 'Add a README file' para que GitHub cree el README automáticamente.

Paso 6 – Hacer clic en 'Create repository'.

Paso 7 – Subir el archivo inventario.py: clic en 'Add file' → 'Upload files' → seleccionar el archivo → 'Commit changes'.

Paso 8 – Editar el README.md con la información del proyecto (ver sección siguiente).

Paso 9 – Copiar la URL del repositorio (ejemplo: <https://github.com/TuUsuario/programa-python>) e incluirla en el informe PDF.

10. Archivo README.md

El siguiente contenido debe copiarse exactamente en el archivo README.md del repositorio GitHub:

```
# 📦 Sistema de Auditoría de Inventario y Reabastecimiento

**Autor:** Emmanuel Orellano Villazón
**Curso:** Fundamentos de Programación - Código 213022
**Universidad:** UNAD - Ingeniería de Sistemas
**Fase:** 5 - Evaluación Final POA - 2026

---

## 📄 Descripción del Problema

Sistema que audita el inventario de una bodega para determinar
qué artículos necesitan ser reabastecidos, calculando la cantidad
exacta a pedir para cada uno.

## 📁 Estructura del Repositorio

```
.
├── programa-python/
│ ├── inventario.py ← Código fuente principal
│ ├── README.md ← Este archivo
│ └── docs/
│ └── informe.pdf ← Informe completo de la solución
└── ...
```

## ▶️ Cómo ejecutar

```bash
python inventario.py
```

## 🔗 Estructura del Código



Función	Descripción
`calcular_pedido()`	Calcula unidades a pedir (función principal)
`clasificar_estado()`	Etiqueta el estado del stock
`generar_informe()`	Recorre la matriz e imprime el informe
`mostrar_lista_pedidos()`	Muestra solo artículos con pedido
`main()`	Punto de entrada del programa



## ✅ Estructuras utilizadas
```



```
- **Listas anidadas:** matriz `inventario[][]`  
- **Funciones:** `def calcular_pedido()`, `def main()`, etc.  
- **Condicionales:** `if`, `elif`, `else`  
- **Repetición:** `for articulo in inventario`
```

11. Conclusiones

Conclusión 1 – Uso de Funciones

La implementación de la función `calcular_pedido()` demuestra la utilidad de la programación modular: una sola función encapsula toda la lógica de negocio del problema (comparar stock actual vs. mínimo y calcular la diferencia), lo que permite reutilizarla para cada artículo de la matriz sin duplicar código. Esto evidencia el principio de abstracción que es fundamental en el desarrollo de software profesional.

Conclusión 2 – Estructuras Condicionales

Las estructuras `if`, `elif` y `else` son indispensables para implementar la toma de decisiones en un programa. En esta solución se utilizan en dos funciones: `calcular_pedido()` para determinar si se genera pedido, y `clasificar_estado()` para asignar una etiqueta descriptiva al stock. La diferencia entre `if/else` (dos caminos) y `if/elif/else` (múltiples caminos) se evidencia claramente en el código.

Conclusión 3 – Ciclos For y Matrices

El ciclo `for` es la estructura más eficiente para recorrer listas anidadas (matrices) en Python. Al combinar el ciclo `for` con el acceso por índices (`articulo[0]`, `articulo[1]`, etc.), se procesa automáticamente cualquier número de artículos sin necesidad de código adicional. Esto demuestra cómo los ciclos eliminan la repetición manual de instrucciones y hacen el código escalable.

Conclusión 4 – Listas Anidadas como Matrices

Python no tiene un tipo de dato nativo para matrices como otros lenguajes, pero las listas anidadas (listas de listas) cumplen perfectamente esta función. La solución implementada demuestra cómo crear, recorrer y extraer datos de una lista anidada de forma eficiente, lo que es aplicable a una gran variedad de problemas del mundo real como sistemas de inventario, bases de datos en memoria o tablas de datos.

Conclusión 5 – Buenas Prácticas de Programación

El código desarrollado aplica buenas prácticas profesionales: uso de comentarios descriptivos, nombres de variables en español que explican su propósito, separación en funciones con responsabilidad única, docstrings en cada función, y uso del bloque `if __name__ == '__main__':` que permite que el código sea reutilizable como módulo. Estas prácticas son valoradas en la industria del software y en la evaluación académica.

12. Referencias

- Ceballos Sierra, F. J. (2015). C/C++. Curso de programación (4.^a ed.). RA-MA Editorial. <https://elibro-net.bibliotecavirtual.unad.edu.co/es/ereader/unad/106454>
- Deitel, P., & Deitel, H. (2016). Cómo programar en C/C++. Pearson Educación. <https://www-ebooks7-24-com.bibliotecavirtual.unad.edu.co/?il=16931&pg=145>
- GitHub, Inc. (2025). Acerca de GitHub y Git. GitHub Docs. <https://docs.github.com/es/get-started/start-your-journey/about-github-and-git>
- Zambrano, J. P. (2025). Introducción al uso de GitHub [Objeto virtual de información]. Repositorio Institucional UNAD. <https://repository.unad.edu.co/handle/10596/75876>